

Data Sampling & Profiling

Sampling

Some Theory

Practical Way

Sampling with Spark

Standard API

Custom Implementation

Distribution

Statistical Characteristics

Deequ for Profiling

Analyzers

Checks

Links

Sampling

Some Theory

Sample recruitment is ongoing. The goal is to recruit in areas with fast-changing signals (hot spots) or high noise or that are currently underrepresented to improve the quality of sample estimates. Those areas and tasks have to be identified on the fly using information from the current participants.

Some incoming samples are best omitted. This is, in a sense, the inverse of the sample recruitment challenge just mentioned: how to decide when to exclude a participant or device from the study, perhaps because it produces only redundant, noisy, spotty, or perverse data.

Participants will enter and leave the sensing program at different times. This hints of panel sampling with informative drop-out, but with much less structure. Note that the problem is non-stationary when the goals change over time or the user groups that are interested in a fixed goal change over time.

Data need not be taken from all available devices all the time. Randomly sampling available devices is one option, but that could be unwieldy if it requires augmenting information from one device with information from others.

Data obtained from participatory sensing and crowdsourcing is likely to be biased. The mix of participants probably does not at all resemble a simple random sample. Those who are easiest to recruit may have strong opinions about what the data should show, and so provide biased information. Samples may need to be routinely re-weighted, again on the fly, with the weights depending on the purpose of the analysis. New ways to describe departures from random sampling may need to be developed. Past work on observational studies may be relevant here.

Practical Way

- Sample should be based on new and old data both, but
 - We need to take more samples from newer data. Older data (in context of Data Quality is already validated)
- Sample outside of the maximum range of distribution.
 - Basically pick more outliers in the sample

Sampling with Spark

Standard API

- **Random Sampling:** Use `.sample(fraction=0.1)` to get a random fraction of rows.
- **Sampling with Replacement:** Use `.sample(withReplacement=True)` to allow the same row to be selected multiple times.
- **Stratified Sampling:** Use `.sampleBy()` based on a specific column for stratified sampling.
 - `fractions = {"A": 0.2, "B": 0.1}`
 - `sampled_data = data.stat.sampleBy("category", fractions, seed=42)`
- **Reservoir Sampling:** Use `.limit(100)` to sample exactly 100 rows, or use `sample()` with a calculated fraction for approximate sampling.
- **SQL Sampling:** Use SQL's `TABLESAMPLE` clause for a SQL-based sampling approach.
 - `sampled_data = spark.sql("SELECT * FROM my_table TABLESAMPLE(10 PERCENT)")`
- **Combining Filters:** Combine filtering with sampling for more specific sampling criteria.

Custom Implementation

- Based on quantiles of data for non-normal distribution

Sample

```
quantiles = data.approxQuantile("value", [0.05, 0.95], 0.01)
lower_quantile = quantiles[0]
upper_quantile = quantiles[1]

# Filter for outliers based on quantiles (values below the 5th percentile or above the 95th percentile)
outliers = data.filter((data['value'] < lower_quantile) | (data['value'] > upper_quantile))

# Optionally, sample the outliers (e.g., take 20% of the outliers)
sampled_outliers = outliers.sample(fraction=0.2, seed=42)
```

 Copy to clipboard

- Outside of 3 standard deviations from mean for normal distribution

Stddev

```
stats = data.select(
    F.mean('value').alias('mean_value'),
    F.stddev('value').alias('stddev_value')
).collect()[0]

mean_value = stats['mean_value']
stddev_value = stats['stddev_value']

# Define the outlier threshold (e.g., more than 3 standard deviations from the mean)
outlier_lower_bound = mean_value - 3 * stddev_value
outlier_upper_bound = mean_value + 3 * stddev_value

# Filter for outliers (values beyond 3 standard deviations from the mean)
outliers = data.filter((data['value'] < outlier_lower_bound) | (data['value'] > outlier_upper_bound))

# Optionally, you can sample the outliers further (e.g., take 20% of the outliers)
sampled_outliers = outliers.sample(fraction=0.2, seed=42)
```

 Copy to clipboard

Distribution

Statistical Characteristics

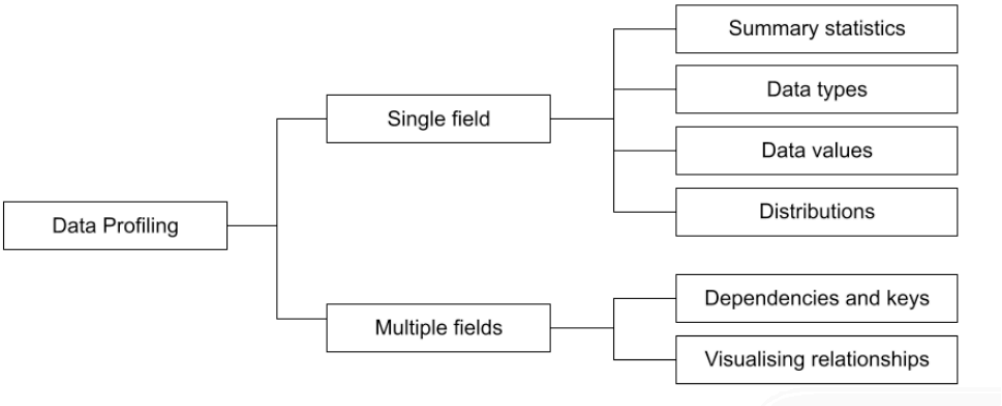
- Data is expected to come from a given distribution.
- The average, variance, or median value is expected to be within a given range with high probability.

Statistical tests still require us to know what to expect, but expectation now has a different form:

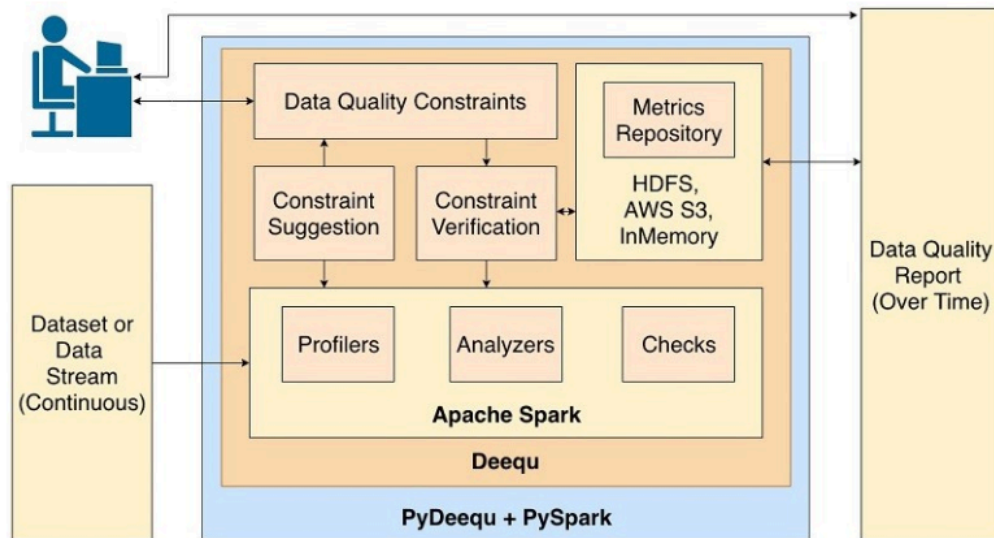
- Is the probability that this data comes from this distribution high enough?
- The average of this column should be within this range with 95% probability.

Analysis

- Expectation and variance of values in a given column
- Distribution of the values in a given column, e.g., the number of events per minute of the day.
- Look for features of the data and their expected distribution.
- A certain seasonality over time is expected, e.g., a spike during payment cycle and less traffic on weekends.
- Run anomaly detection algorithms to look at the history of features and see if the current values are normal



Deequ for Profiling



- Download the jar
 - `s3://data-pipeline-orchestrator/elasticmapreduce/external-jars/deequ-2.0.7-spark-3.3.jar`
- Install pydeequ
 - `pip install pydeequ`
- Set env variable
 - `export SPARK_VERSION=3.3`
- Submit spark job

Spark submit

```

spark-submit
--deploy-mode client
--conf spark.port.maxRetries=60
--conf spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension
--conf spark.dynamicAllocation.maxExecutors=5
--conf spark.dynamicAllocation.enabled=true
--jars s3://data-pipeline-orchestrator/elasticmapreduce/external-jars/deequ-2.0.7-spark-3.3.jar
--conf spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog s3://quad-delta-lake-bronze

```

Copy to clipboard

- Pyspark script

Code

 Copy to clipboard

```
from pyspark.sql import SparkSession
from pydeequ.analyzers import *
from pydeequ.repository import *

import datetime
import json
import os

import boto3
import psycpg2
import pyspark.sql.functions as F
from botocore.exceptions import ClientError
from delta.tables import *
from pyspark.sql import DataFrame, SparkSession # type: ignore
from pyspark.sql.functions import *

os.environ['SPARK_VERSION'] = '3.3'

spark_session_id_name = 'deequ_test'
spark_session_id = spark_session_id_name + spark_session_id_name
spark = SparkSession.builder.appName(spark_session_id).getOrCreate()

# Load some sample data (from S3, HDFS, or local file system)
data = spark.sql("Select * from lmsdb_gold.advance_payment_pg where year = 2024 and month = 9 and day = 21 ")

# Define the analyzers for gathering data distributions
# You can analyze column data types, histograms, etc.

analysis_result = AnalysisRunner(spark)\
    .onData(data)\
    .addAnalyzer(ApproxCountDistinct("principal_advance_payment"))\
    .addAnalyzer(Minimum("principal_advance_payment"))\
    .addAnalyzer(Maximum("principal_advance_payment"))\
    .addAnalyzer(Mean("principal_advance_payment"))\
    .addAnalyzer(StandardDeviation("principal_advance_payment"))\
    .addAnalyzer(ApproxQuantiles("principal_advance_payment", quantiles=[0.25, 0.5, 0.75]))\
    .addAnalyzer(Completeness("principal_advance_payment"))\
    .addAnalyzer(ApproxCountDistinct("interest_savings"))\
    .addAnalyzer(Minimum("interest_savings"))\
    .addAnalyzer(Maximum("interest_savings"))\
    .addAnalyzer(Mean("interest_savings"))\
    .addAnalyzer(StandardDeviation("interest_savings"))\
    .addAnalyzer(ApproxQuantiles("interest_savings", quantiles=[0.25, 0.5, 0.75]))\
    .addAnalyzer(Completeness("interest_savings"))\
    .addAnalyzer(Completeness("id"))\
    .addAnalyzer(Completeness("created_at"))\
    .run()

analysisResult_df = AnalyzerContext.successMetricsAsDataFrame(spark, analysis_result)
analysisResult_df.show(50,False)
```

• Results

Results

Column principal_advance_payment ApproxQuantiles-0.25 1.04E8	
Column principal_advance_payment ApproxQuantiles-0.5 5.04E8	
Column principal_advance_payment ApproxQuantiles-0.75 1.69923E9	
Column id Completeness 1.0	
Column interest_savings Minimum 0.0	
Column interest_savings Maximum 2.4219E10	
Column interest_savings Completeness 1.0	
Column interest_savings ApproxQuantiles-0.25 1.735E7	
Column interest_savings ApproxQuantiles-0.5 8.286E7	
Column interest_savings ApproxQuantiles-0.75 3.2718E8	
Column principal_advance_payment Minimum 10000.0	
Column principal_advance_payment Maximum 8.874E10	
Column principal_advance_payment Completeness 1.0	
Column created_at Completeness 1.0	
Column principal_advance_payment ApproxCountDistinct 3505.0	
Column principal_advance_payment Mean 1.9367056712390375E9	
Column principal_advance_payment StandardDeviation 4.503137429229805E9	
Column interest_savings ApproxCountDistinct 3531.0	
Column interest_savings Mean 3.692111738250506E8	
Column interest_savings StandardDeviation 9.414476014016665E8	

 Copy to clipboard

Analyzers

- ApproxQuantile
- ApproxQuantiles
- Completeness
- Compliance
- Correlation
- CountDistinct
- DataType
- DataTypeInstances
- Distinctness
- Entropy
- Histogram
- KLLParameters
- KLLSketch
- MaxLength
- Maximum
- MinLength
- Minimum
- MutualInformation
- Size
- StandardDeviation
- Sum
- UniqueValueRatio
- Uniqueness

Checks

A lot of checks based on analyzers, like

- hasCompleteness
- hasApproxQuantile
- hasCorrelation
- hasDataType
- hasDistinctness
- hasMax
- hasMaxLength
- hasMean

Links

- https://pydeequ.readthedocs.io/_/downloads/en/latest/pdf/